

Design Standards & Guidelines for 360° Extensions and Integrations

Contents

Introduction and background

General guidelines

Coding and quality guidelines

No admin privileges

Logging

Package.xml tasks and attributes

Guidelines for quality and security checks in custom Git repositories

Branch policies

Security controls

Integrations

AuthProxy - What it is and how to configure it

SOAP Integrations and Services

Extend Business Logic using RPC REST

Modern application hosting

Schema/package import

Outbound integrations

BIF customizations

Database modifications

Process Engine workunits and jobs

Standalone/delivery package design guidelines

Copy DLLs to bin folder, not GAC

Avoid using webscripts tasks

Never depend on anything persistent on the local disk

"XCopy deployment"

Package template

Introduction and background

This article is a work in progress and will be continuously updated as we proceed.

In this document we describe the different ways we support extending 360° in the future. For on-premises, we recommend these guidelines to be followed from 5.0 onwards. For 360° Online, we **require** these guidelines to be followed already.

Moving forward, our main focus is to design solutions that will work well in a containerized environment, but it's important to remember that containers is not the end-goal, it just a step in the evolution. Today we must deal with configuring containers, managing Kubernetes and so on. In the future we probably on upload an application and the infrastructure will take care of

deployment and resource allocation – much like serverless works today for individual functions. Its our duty as developers and architects to try to ensure that what we design and build will be as future proof as possible (in addition to be well crafted of course).

For 360° Online, where we cannot have huge upgrade projects for scalability reasons, we require the standards and guidelines below to be followed even now. **Packages that does not follow these guidelines will be rejected!**

General guidelines

Coding and quality guidelines

Coding and quality guidelines are covered in a different wiki: [Coding and Quality Guidelines for Development](#)

No admin privileges

360° Online does not run with service users with local administrator privileges. This means that any operation that is done within a custom DLL **MUST** assume that the operations are run as a regular user, or even an ApplicationPoolIdentity user (see <https://docs.microsoft.com/en-us/iis/manage/configuring-security/application-pool-identities> for more information about these users).

It is also strongly recommended, for security reasons, that also on-premises installations use service users that are NOT administrator!

Logging

For logging, please see coding standards for development: [Coding Standards and Quality Requirements for Development#Logging](#)

Package.xml tasks and attributes

With containers, delivery packages can still be installed - BUT - the full syntax of the package.xml schema is NOT supported. Only use the parts of the schema that are supported in our container implementation. This is listed here: [360° Docker Containers#package-install](#)

If it's not on this list, **DO NOT** use it in your delivery package. Do also note that this list will likely expand as we go on. And if you have a request for something that you really think should be supported, please send kristina.devochko@tietoevry.com an e-mail.

Guidelines for quality and security checks in custom Git repositories

When extensions or integrations require a separate repository with source code and build pipeline to be created in Azure DevOps, it's important to ensure that the same code quality, code review and security policies as in standard 360 repositories are implemented in custom repository. It is

important to continuously review and update custom repositories in case some of the controls are missing! In this way we ensure that all the code, both standard and custom, that is being delivered to our customers has the same high quality and security level.

Mandatory checks that must be in place in every repository using a build pipeline.

Branch policies

Master branch must have proper branch policies in place:

- All changes must be added via pull requests i.e. no direct push must be allowed to master
- Build must execute and pass on every pull request
- Reviewers must be added on every pull request. There should be at least one reviewer apart from the PR author and if the code will be included in standard 360 setup, core-trusted-reviewers-bl and/or core-architects-and-techleads reviewer group must be added. All reviewers must follow standard procedure for code review described in Code review requirements.

Security controls

We have created a build pipeline template that you can use - it will include all security build tasks implemented so that you can easily re-use it.

Following security controls must be implemented in build pipeline:

- **Scanning of third-party dependencies must be in place:** for frontend **npm audit** must be used and for backend **OWASP Dependency Check** must be used. For OWASP Dependency Check you will need to create a suppression file in the root of the repository called "owasp-depcheck-suppression.xml".
- **Code signing must be implemented for all custom assemblies.** First of all, you will need to add "core-build" variable group to variables section of the pipeline definition
- **Static code analyzer (SAST tool) should be implemented in order to statically analyze your code for insecure patterns and known vulnerabilities.** For .NET code we're using Security Code Analyzer:
- **Validation for assemblies with missing code signing must be implemented.** We must not add third-party packages that don't have code signing in place. Therefore it's important to have a validation implemented as part of the build pipeline that will scan for unsigned assemblies and raise an alert in case new unsigned assemblies are being added. For unsigned assemblies validation you will need to create a whitelist file in the root of the repository called "unsigned-dll-whitelist.txt".

There are many standard repositories that have these validations already implemented in the build pipeline, you're welcome to re-use those tasks in your own pipeline!

"core" repository where we have source code for all microservices is a good starting point.

For "npm audit", please see "npm audit" build task for "teams-react-app" microservice: [teams-react-app \(https://tieto-si.visualstudio.com/360/_git/core?path=%2Fweb%2Fmicroservices%2Fteams-react-app%2Fazure-pipelines.yml\)](https://tieto-si.visualstudio.com/360/_git/core?path=%2Fweb%2Fmicroservices%2Fteams-react-app%2Fazure-pipelines.yml)

For OWASP Dependency check and Security Code Analyzer for static code analysis, please see "Application_Security_Validation" build job in "matrikkel-adapter" microservice: [matrikkel-adapter \(https://tieto-si.visualstudio.com/360/_git/core?path=%2Fweb%2Fmicroservices%2Fmatrikkel-adapter%2Fazure-pipelines.yml\)](https://tieto-si.visualstudio.com/360/_git/core?path=%2Fweb%2Fmicroservices%2Fmatrikkel-adapter%2Fazure-pipelines.yml)

For code signing (Digital Sign) and unsigned assemblies validation, please see "MatrikkelAdapter_Build_Image_Linux" build job in "matrikkel-adapter" microservice: matrikkel-adapter (https://tieto-si.visualstudio.com/360/_git/core?path=%2Fweb%2Fmicroservices%2Fmatrikkel-adapter%2Fazure-pipelines.yml) . Important pre-requisite for these build tasks - you need to add "common-git-version-increment" as a submodule to your repository. This submodule includes all the necessary PowerShell scripts that are called from the respective build tasks. Take a look here to find out how to add a submodule to a repository: Add a submodule (https://tieto-si.visualstudio.com/360/_wiki/wikis/Core%20Wiki/15/README?anchor=add-as-submodule)

More information about security requirements can be found in standard coding guidelines Wiki: Coding Standards and Quality Requirements for Development - Software Innovation Wiki (software-innovation.com)

If you have any questions or reflections regarding this, please post them to "Core RnD Development" Teams channel.

Integrations

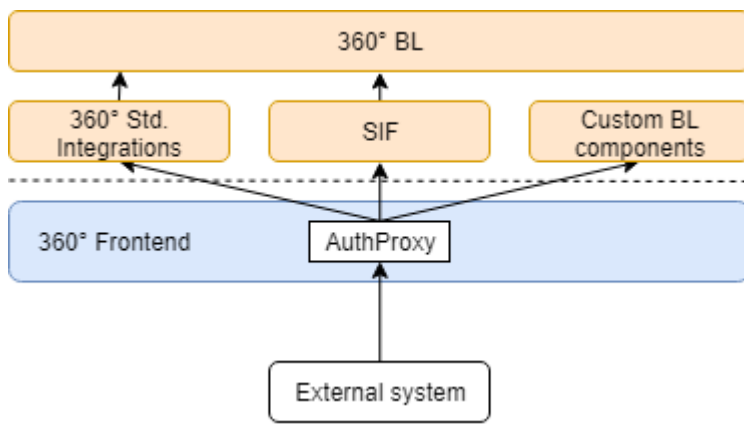
We support integrating with 360° in many ways:

- SOAP endpoints, where an external service invokes an operation in 360° through SOAP. Note that Windows Communication Framework (WCF) is NOT supported.
- REST endpoints, where an external service invokes a REST endpoint in the 360° business logic
- Schema/package import, where a package is uploaded through a REST endpoint in 360°, then processing the package through Process Engine or other means.
- Outbound integrations, where a component run from within the 360° business logic calls out to an external service

Each of these ways of integrating are covered in the sections below. For the two first scenarios, any requests from integrations to **any** backend component (e.g. SIF or the Integration site) **MUST** go through the AuthProxy component. The AuthProxy is a thin layer on top of 360° that handles authentication for server-to-server communication. Additionally, for SOAP services, in order to avoid having to set up a new IIS web application for the SOAP service, we require any SOAP based service to have a translation layer that does NOT use Windows Communication Framework (WCF). Details on how this is set up follows.

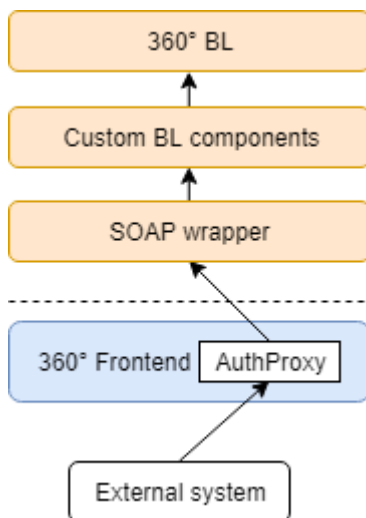
AuthProxy - What it is and how to configure it

Notes: Integration users - how to create them, best practices (no domain user as service user (runas user)).



SOAP Integrations and Services

SOAP based services are very hard to deploy in a container / serverless environment. They typically need a lot of configuration etc. and is also (in the .Net world) typically also based on Windows Communication Framework (WCF). WCF is a dead end that will be phased out over time. For 360° Online, WCF Is also highly undesirable due to its requirement of having a new web application in place in IIS, which requires additional monitoring and overhead during installation. On the other hand, these services often implement public standard and is also supporting a large base of (custom integrations).



Therefore, there is no option to not support SOAP based services in the short run. Instead we have created some helper classes that will make it easy to "fake" a SOAP service with xcopy deployment. No config what so ever!

Let's look at a small sample: The "NorGeoIntegrasjon" (GI) is a Norwegian standard the is defined with a SOAP interface. Today this service is exposed under /integration/gi11.svc and is implemented with WCF.

1. Since we are requiring xcopy deployment, you need to package your bridge implementation in a way that makes it external callable.
2. For service documentation, we need to expose WSDL generation
3. As far as possible – don't modify your underlying service implementation (if you add the support on existing) to support both the old and the new way for some time

See the section on RPC REST for instruction on self-contained xcopy deployment. Create the method that will receive the SOAP message and expose it:

```

1 namespace SI.Biz.Core.SoopBridge
2 {
3     [SurfaceCalllable]
4     public class ArkivInnsyn
5     {
6         public static HttpResponseMessage Process(HttpRequestMessage requestMessage)
7         {
8             return Invoker.Process(
9                 requestMessage,
10                 new
11 Uri("http://rep.geointegrasjon.no/Arkiv/Innsyn/xml.wsd1/2012.01.31/giArkivInnsyn20120131.wsd1"),
12                 "ArkivInnsyn11");
13     }
14 }

```

```

1 namespace SI.Biz.Core.SoopBridge.GI
2 {
3     internal class Invoker
4     {
5         public static HttpResponseMessage Process(HttpRequestMessage requestMessage, Uri wsdlUrl,
6 string lastNsPart)
7         {
8             const string NAMESPACE =
9 "http://rep.geointegrasjon.no/Arkiv/Felles/xml.schema/2012.01.31";
10
11             if (BridgeSupport.GetQueryParameter<bool>(requestMessage, "wsdl", false))
12             {
13                 return BridgeSupport.GetWSDLContent(wsdlUrl).Result;
14             }
15
16             var mf = new SI.Biz.Core.Integration.GI.Svc.Impl11();
17
18             var action = SoapHelper.GetAction(requestMessage);
19             var body = requestMessage.Content.ReadAsStringAsync().Result;
20
21             var ns = "SI.Biz.Core.Integration.ExternalContracts";
22             var inputType = Type.GetType(string.Format("{0}.GI.{1}.{2},{3}", ns, lastNsPart, action,
23 ns));
24             var returnType = Type.GetType(string.Format("{0}.GI.{1}.{2}Response,{3}", ns, lastNsPart,
25 action, ns));
26
27             return SoapHelper.DispatchSoapRequest(
28                 action,
29                 body,
30                 inputType,
31                 returnType,
32                 mf,
33                 NAMESPACE,
34                 wrapped: true,
35                 useDefaultExceptionHandler: true);
36         }
37     }
38 }

```

Extend Business Logic using RPC REST

<TBD> THOR

Modern application hosting

<TBD>

Schema/package import

Outbound integrations

BIF customizations

BIF customizations are not allowed. Please see: [Configuration Possibilities in 360°](#) For custom fields, there's no longer any need for custom BIF. Detail views are also highly customizable now, so there should be minimal need for custom BIF for detail views as well.

TBD - what makes sense??

Database modifications

Any changes to the database would be considered a customization and is not allowed. Please bring up your need for discussion with your project manager, delivery manager, line manager to decide on how to proceed if this is really needed.

Process Engine workunits and jobs

Please visit below link in order to get started with basic understanding of process engine concepts.

<https://wiki.software-innovation.com/wiki/ProcessEngine>

In order to create progress plan from 360 UI and configuration of the same can be done from **360 web admin -> configuration -> process management** section in **5.0**.

Steps to follow if we don't see a workflow combobox in Process template:

There must be at **least one WU created in Process Engine solution**, then only it will be available as selectable process in combobox while creating new template for Progress plan.

For case : We have a codetable with name **CaseType** and we have to make sure that **hasprogressplan** value is set to 1.

For Project : We have a codetable with name **ProjectType** and we have to make sure that **hasprogressplan** value is set to 1.

We should select type of logging (**database/eventlog**) for 360 jobs in order to keep track of business flow (what went wrong) from **360 web admin -> monitoring-> Jobs section**.

Set result type of job.

Standalone/delivery package design guidelines

In this section we'll cover

Copy DLLs to bin folder, not GAC

DLLs that are extensions to the business logic should be copied to the instance's bin folder. Typically this folder is: **C:\Program Files (x86)\Tieto\360_instances\<instance name>\webservices\SI.WS.Core\bin**

Avoid using webscripts tasks

Tasks of type webscripts are not supported in containers and should not be used for any new code. The recommended way of handling these kinds of web.config or XML file changes is to use PowerShell. Example:

TODO

Never depend on anything persistent on the local disk

A container is immutable and will be restored each time the container starts. Therefore any logging done to files or events written to the eventlog will be lost on restart.

THOR: Alternatives? :)

"XCopy deployment"

Since the container is immutable, it does not give any meaning to create new IIS sites and applications inside a running container on start-up since we want the start-up time to be as fast as possible. One other option could be to create new container images after the changes have been made, but this will quickly become unmanageable since we then need to maintain a lot of different images. One image for each permutation of configuration. To solve this, we want to create a container bootstrapper that during start-up performs some simple and fast initialization before starting the application. This includes things like minor config changes and file copy (dll's and workunits). No setups and IIS scripting are allowed. The implications of this is that developers working with 360 needs to adhere to those restrictions, and there are already a lot of supporting functionality that will help to achieve this.

Package template

A delivery package template can be found in Azure DevOps, documentation here: [360 Delivery Template](#)

Retrieved from "https://wiki.software-innovation.com/index.php?title=Design_Standards_%26_Guidelines_for_360°_Extensions_and_Integrations&oldid=55765"

This page was last edited on 12 November 2025, at 11:21.